Vellore Institute Of Technology

OPEN SOURCE SOFTWARE (LINUX ADMINISTRATION)

Course Code: CSE0002

THE OPEN SOURCE AUDIT (*A Capstone Project Report)*

Report Topic:

"Case Study on Python Programming Language"

Submitted By:
Name : Bhagyashree Sethiya
Registration Number: 24BAI10581
Slot: A24

Date of Submission: 28/03/2026

PART-A

A1 — The Problem Python Was Created to Solve

Background Before Python

Before Python was introduced, most programming was done using languages like C and C++. These languages were powerful and allowed developers to control low-level operations, making them suitable for system programming.
However, they were not easy to use. Writing code required strict syntax, careful memory management, and deep technical knowledge. Even small mistakes could lead to errors that were difficult to debug. As a result, beginners often found programming complicated and time-consuming.
With the rapid growth of technology, there was a need for a programming language that could simplify coding and make development faster while still being flexible.

Why Python Was Created

In the late 1980s, Guido van Rossum was working in the Netherlands and had experience with a language called ABC, which aimed to simplify programming. Although ABC introduced useful ideas, it was not suitable for real-world applications.
This led him to design a new language that focused on simplicity and usability. In 1991, Python was created with the goal of being easy to read, easy to write, and useful for practical applications.

Problems Python Solved

1. Readability

Python introduced a clean and simple syntax, making code easier to understand. This improved collaboration among developers.

2. Faster Development

Python is an interpreted language, meaning programs can run directly without compilation. This makes testing and debugging faster.

3. Beginner-Friendly

Python reduced complexity, making programming accessible to beginners. It is now widely used as a first programming language.

Role of Open Source

Python is open source, meaning anyone can use, modify, and distribute it freely. This allows developers worldwide to contribute to its improvement.
Because of this collaborative approach, Python continues to evolve, becoming more reliable and feature-rich over time.

Impact of Python

Today, Python is one of the most popular programming languages. It is used in areas such as web development, automation, data science, and artificial intelligence.
Its simplicity makes it ideal for beginners, while its powerful libraries make it suitable for professionals. This balance has made Python an essential tool in modern computing.

In my opinion, Python became popular mainly because it removes the fear of coding for beginners and allows faster experimentation.

A2 — The License: What It Actually Says (Python)

Understanding Software Licensing

Software licensing defines the rules under which a program can be used, modified, and shared. In traditional proprietary software, users usually do not get access to the source code. They can only use the software under strict conditions, and modification or redistribution is not allowed. This keeps full control with the company that owns the software.

In contrast, open-source software follows a more flexible approach. It allows users to access the source code, understand how it works, and even modify it. At the same time, certain conditions are applied, such as keeping the original copyright notice. This ensures that the original creators receive proper credit.

Python's License and Its Freedoms

Python Software Foundation License is the license under which Python is distributed. It is a **permissive open-source license**, meaning it gives users a lot of freedom with very few restrictions.

Under this license:

- Python can be used freely for personal, academic, and commercial purposes
- There is no cost involved in using the language
- Developers can modify the source code
- Modified versions can also be shared

The only important condition is that the original license and copyright notice must be included.

These permissions match the idea of the **four freedoms of free software**:

1. Freedom to use the software
2. Freedom to study how it works
3. Freedom to modify it
4. Freedom to distribute copies

Python satisfies all of these, making it a true open-source system.

Can Companies Modify and Sell Python?

Yes, companies can use Python, modify it, and even include it in their products. They are allowed to sell those products as well. However, they must include the original license and give proper credit.

Unlike some stricter licenses, companies are **not forced to share their modifications publicly**. This makes Python very attractive for commercial use.

Comparison with Other Licenses

- GNU General Public License (GPL):
  This is a **copyleft license**. If someone modifies the software, they must release their changes under the same license. This ensures everything stays open-source, but companies may avoid it.
- MIT License:

This is very flexible with minimal restrictions. The PSF License is quite similar, as both allow easy use and integration.

Compared to GPL, Python's license is more flexible. Compared to MIT, it is quite similar in spirit.

Free as in Cost vs Free as in Freedom

There is an important difference between:

- **Free as in cost** → no money required
- **Free as in freedom** → control over how software is used

Python satisfies both. It is free to download, but more importantly, it allows users to modify and adapt it according to their needs.

In my opinion, this freedom is more valuable because it encourages innovation and reduces dependence on a single company.

A3 — The Ethics of Open Source Software

Should All Software Be Open Source?

Open-source software is often seen as an ideal because it promotes transparency, collaboration, and shared knowledge. When software is open source, anyone can study the code, modify it, and improve it. This helps developers learn from each other and build better systems faster. It can also improve security, since many people can review the code and identify problems.

However, it is not practical for all software to be open source. Some companies invest a lot of time and money into developing their products, and they rely on keeping their code private to protect their work. In areas like business applications or sensitive systems, keeping software proprietary can be necessary for security and competition.

In my opinion, open source is very useful and should be encouraged, but not all software can or should be open. A balance between open-source and proprietary models is more realistic.

Is It Ethical for Companies to Profit from Open Source?

Many companies build profitable products using open-source software. This is legally allowed, especially under permissive licenses. In fact, open source forms the base of many modern technologies used by big companies.

From one side, this is positive because companies help spread and improve open-source tools. Some organizations also contribute back by fixing bugs, improving performance, or releasing new tools.

On the other hand, there is an ethical concern. If companies earn huge profits using open-source projects but do not contribute back, it may feel unfair to the developers who created those projects. Open-source communities often depend on voluntary contributions, and lack of support can affect their sustainability.

According to me, even if it is not legally required, companies should contribute back in some way. This could be through code, funding, or community support.

Does Open Source Help or Limit Innovation?

The idea of "standing on the shoulders of giants" means that new innovations are built on existing knowledge. Open source strongly supports this idea by providing a base that developers can build upon.

Because of open source, developers do not need to start from scratch every time. They can reuse

existing tools and focus on creating new features or solving bigger problems. This speeds up development and encourages creativity.

Some people argue that relying too much on existing tools may reduce originality. However, in most cases, open source actually increases innovation by making development easier and more accessible.

Overall, I believe open source acts as an enabler of innovation. It allows more people to participate in development and helps technology grow faster.

PART B — LINUX FOOTPRINT

Python in Linux Systems

Python is an important part of most Linux systems and is often pre-installed in many distributions. Unlike normal software that needs separate installation, Python is already integrated into the system because it is used in many internal operations.

Many Linux tools and system utilities depend on Python for tasks like package management, automation, and configuration. Because of this, Python is not just a programming language but also a core component of the Linux environment.

Installation Method

Python is usually installed using Linux package managers such as:

- apt (Debian/Ubuntu-based systems)
- yum or dnf (Red Hat-based systems)

For example, in Debian-based systems, Python can be installed or verified using:

sudo apt install python3

This command either installs Python or confirms that it is already present in the system. Package managers automatically handle dependencies, making installation simple and efficient.

```
userbhagyashree@Ayushi:~/python-oss-audit$  sudo apt install python3
[sudo] password for userbhagyashree:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
userbhagyashree@Ayushi:~/python-oss-audit$ python3 --version
Python 3.12.3
userbhagyashree@Ayushi:~/python-oss-audit$
```

Multiple Versions Management

One important feature of Linux is that it allows multiple versions of Python to exist at the same time.

- One version is used by the system (for internal processes)
- Another version can be used by the user for development

This separation prevents system errors and ensures stability even if users install or update different versions. It shows how Linux manages software efficiently without conflicts.

Verification of Installation

After installation, Python can be verified using commands like:

python3 --version

This command shows the installed version of Python.

Another useful command is:
which python3
This tells the exact location of the Python executable in the system.

```
userbhagyashree@Ayushi:~/python-oss-audit$ python3 --version
Python 3.12.3
userbhagyashree@Ayushi:~/python-oss-audit$ which python3
/usr/bin/python3
userbhagyashree@Ayushi:~/python-oss-audit$ ls /usr/bin/python3
/usr/bin/python3
userbhagyashree@Ayushi:~/python-oss-audit$
```

Directory Structure and Integration

Python is usually located in standard Linux directories such as:

- /usr/bin/python3 → main executable
- /usr/lib/python3/ → libraries
- /etc/ → configuration (if applicable)

These directories show how Python is organized within the Linux file system and how it integrates with other system components.

User and System Role

Python programs generally run under the current logged-in user. However, system-level Python processes may run under specific system users for security reasons.
This separation helps protect the system and ensures that user actions do not affect critical system operations.

File System Structure, Users, and Permissions

File System Organization of Python in Linux

In Linux, software is not stored in a single folder. Instead, it is distributed across different directories based on its purpose. Python also follows this structured approach.

The main Python executable is usually located at:

- /usr/bin/python3 → contains the Python interpreter

The standard libraries are stored in:

- /usr/lib/python3/ → contains built-in modules and libraries

This separation helps keep the system organized and easy to manage. Binaries, libraries, and configuration files are placed in different locations, which improves clarity and maintainability.

When users install additional Python packages using tools like pip, they are often stored in user-specific directories (such as inside the home folder). This prevents conflicts with system-level files and ensures stability.

```
userbhagyashree@Ayushi:~/python-oss-audit$ ls /usr/bin/python3
/usr/bin/python3
userbhagyashree@Ayushi:~/python-oss-audit$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
userbhagyashree@Ayushi:~/python-oss-audit$
```

User Environment and Access Control

Linux is a multi-user operating system, meaning multiple users can work on the same system with controlled access. Python works within this system by following user permissions.

When a Python program runs, it uses the permissions of the current user. This means:

- It can only access files the user is allowed to access
- It cannot modify restricted system files

For system-level tasks (like installing packages globally), administrative privileges are required using commands like:

sudo apt install python3

Regular users can still install packages in their own environment without affecting the system. This separation improves both security and flexibility.

```
userbhagyashree@Ayushi:~/python-oss-audit$ whoami
userbhagyashree
userbhagyashree@Ayushi:~/python-oss-audit$
```

Permission System in Linux

Linux uses a permission system based on:

- **Read (r)** → view file contents
- **Write (w)** → modify file
- **Execute (x)** → run file

These permissions are assigned to:

- Owner
- Group
- Others

In Python, execute permission is important for running scripts. For example:

chmod +x script.py

After this, the script can be executed directly from the terminal.

```
userbhagyashree@Ayushi:~/python-oss-audit$ cd ~
userbhagyashree@Ayushi:~$ pwd
/home/userbhagyashree
userbhagyashree@Ayushi:~$ touch script.py
userbhagyashree@Ayushi:~$ ls -l script.py
-rw-r--r-- 1 userbhagyashree userbhagyashree 0 Mar 27 09:42 script.py
userbhagyashree@Ayushi:~$ chmod +x script.py
userbhagyashree@Ayushi:~$ ls -l script.py
-rwxr-xr-x 1 userbhagyashree userbhagyashree 0 Mar 27 09:42 script.py
userbhagyashree@Ayushi:~$
```

Practical Importance of Permissions

Permissions control how Python interacts with the system. For example:

- A script cannot modify files in /usr/bin without admin rights
- Users cannot access each other's private files

This ensures:

- System security
- User isolation
- Prevention of accidental damage

Because of this, Python works safely within Linux without affecting critical system components.

Execution, Management, and Update Mechanisms

Execution Model of Python in Linux

Python works in Linux as an **interpreted language**, which means programs are executed only when required. Unlike services such as web servers or databases that run continuously in the background, Python scripts run only when the user executes them.

A Python program is usually executed using a command like:

python3 script.py

This on-demand execution makes Python very flexible and efficient, as it does not consume system resources when not in use.

```
userbhagyashree@Ayushi:~$ echo "print('hello linux')" > script.py
userbhagyashree@Ayushi:~$ python3 script.py
hello linux
userbhagyashree@Ayushi:~$
```

Python scripts can also be made executable like normal Linux commands by:

Adding a **shebang line** at the top of the file (#!/usr/bin/python3)

Giving execute permission chmod +x script.py

After this, the script can be run directly:

./script.py

This shows how well Python integrates into the Linux command-line environment.

Package Management and Dependency Handling

Python in Linux is managed using both:

- **System package managers** (like apt, yum, dnf)
- **Python's own package manager (pip)**

System-level tools (like apt) are used to install and update Python itself. For example:

sudo apt install python3

These tools ensure compatibility with the system and handle dependencies automatically.

```
userbhagyashree@Ayushi:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
userbhagyashree@Ayushi:~$
```

For installing additional libraries, Python provides pip, which allows users to add external packages:

pip install numpy

This separation is very useful:

- System packages → controlled by OS
- Python libraries → controlled by user

It ensures system stability while giving developers flexibility.

Update Mechanisms in Linux

Python updates in Linux are distributed through official repositories and maintained by the open-source community.

Users can update their system using commands like:

sudo apt update

sudo apt upgrade

These updates include:

- Security patches
- Bug fixes
- Performance improvements

```
userbhagyashree@Ayushi:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.3 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Fetched 222 kB in 2s (134 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
54 packages can be upgraded. Run 'apt list --upgradable' to see them.
userbhagyashree@Ayushi:~$
```

Role of Open Source in Updates

Unlike proprietary software, Python is developed and improved by a global community. Developers from different parts of the world contribute code, fix issues, and suggest improvements.

This results in:

- Faster bug fixing
- Continuous improvement
- Better security

At the same time, updates are released in a structured way to ensure stability. This balance between community contribution and controlled releases makes Python reliable for both personal and professional use.

Part C — Ecosystem and Role of Python in Modern Software Systems

Python in the Modern Open Source Ecosystem

The modern software ecosystem is made up of many tools, libraries, and frameworks that work together to build applications. In this ecosystem, Python plays a very important role because of its simplicity and flexibility.

Python is used in many different areas such as:

- Web development
- Data science
- Artificial intelligence
- Automation
- System scripting

This wide usage makes Python a versatile language that connects different parts of software development.

One of Python's biggest strengths is its large collection of open-source libraries and frameworks. For example:

- Django and Flask → used for web development
- NumPy and Pandas → used for data processing
- TensorFlow → used for AI and machine learning

These tools are continuously improved by developers around the world, which keeps Python updated with modern technologies.

Another important factor is strong community support. Python has extensive documentation, tutorials, and online forums. This makes it easier for developers to learn and solve problems quickly.

Comparison with Traditional LAMP Stack

The traditional LAMP stack includes:

- Linux (Operating System)
- Apache (Web Server)
- MySQL (Database)
- PHP (Scripting Language)

This architecture is reliable but limited in flexibility.

Python provides an alternative by replacing PHP with Python frameworks. This leads to a modified stack often called **LAPP (Linux, Apache, PostgreSQL/MySQL, Python)**.

Using frameworks like Django or Flask, developers can build applications that are:

- More structured
- Easier to maintain
- More scalable

Python also allows flexibility in choosing technologies. For example:

- Web servers like Nginx can be used

- Databases like PostgreSQL can be integrated

This flexibility helps developers design systems according to their needs instead of following a fixed structure.

Python in Modern Application Development

Python is widely used in building modern applications because it is easy to write and maintain. Its simple syntax helps developers manage large projects more efficiently.

It is commonly used in:

- Cloud computing
- Automation systems
- Data-driven applications

Python can easily connect with databases, APIs, and other technologies. It can act as a backend language and handle complex operations like data processing and communication between systems.

Another advantage is the availability of frameworks that support:

- Code organization
- Testing
- Deployment

Although Python may be slower than low-level languages like C, its faster development time and ease of use make it more practical in most real-world applications.

Why Python is So Popular

Python has become popular due to several key features:

- Simple and readable syntax
- Large number of libraries and frameworks
- Strong community support
- Cross-platform compatibility
- Easy integration with other technologies

Because of these features, Python is suitable for both beginners and experienced developers.

Part D — Open Source vs Proprietary Software

Dimension	**Open Source (Python)**	**Proprietary Software (e.g., MATLAB)**
Cost	Freely available, no license fee required	Paid software, often expensive
Source Code	Open and accessible to everyone	Closed and restricted
Security	Can be reviewed and improved by community	Depends on company for updates
cdSupport and Reliability	Community forums and documentation	Official company support
Freedom to Modify	Can be modified and redistributed	Very limited or no modification allowed
Control	Developed and maintained by community	Controlled by a single organization

Comparison Between Open Source (Python) and Proprietary Software

Analysis and Discussion

Open-source software like Python provides many advantages, especially in terms of flexibility and accessibility. Since the source code is available, developers can modify it according to their needs. This encourages collaboration, as people from around the world can contribute and improve the software. It also helps in faster innovation because improvements are shared openly.

Another major benefit is cost. Python is free to use, which makes it very useful for students, startups, and research projects where budget is limited.

In terms of security, open-source software can actually be quite strong. Since the code is visible, many developers can review it and identify issues. However, this depends on how active the community is in maintaining the project.

On the other hand, proprietary software like MATLAB provides a more controlled and structured environment. It comes with official support, proper documentation, and regular updates from the company. This makes it reliable for professional and enterprise use.

However, proprietary software has some limitations. Users cannot modify the source code, and they depend completely on the company for updates and improvements. This can lead to vendor lock-in, where switching to another system becomes difficult.

Conclusion and Personal Verdict

In my opinion, open-source software like Python is more flexible and suitable for modern software development. It is cost-effective, easy to use, and supported by a large community. The ability to modify and extend the software makes it very useful in situations where requirements keep changing.

Although proprietary software offers benefits like dedicated support and stability, it often lacks flexibility and can be expensive. These limitations make it less suitable for many modern applications.

Therefore, I would recommend using open-source software like Python in most cases, especially where customization, cost efficiency, and long-term growth are important. Open source not only reduces dependency on a single company but also encourages learning, innovation, and collaboration.

Part E — Implementation Using Shell Scripting

Introduction

This section presents the practical implementation of the project using Linux shell scripting. A menu-driven command-line interface (CLI) is created to combine multiple scripts into a single system.

Each script performs a specific task such as:

- System information retrieval
- File operations
- User interaction
- Process monitoring
- Automation

This implementation shows how open-source tools and basic Linux commands can be used to build useful system-level utilities.

Project Structure

The project follows a **modular structure**, where each script is responsible for a specific function. This makes the system organized, easy to maintain, and scalable.

The main.sh file acts as the **entry point**, allowing users to access all features through a single interface.

1. Main Script (main.sh)

The main.sh script is the central controller of the project. It provides a menu-driven interface where users can choose different operations.

Based on user input:

- A **case statement** is used to select the corresponding option
- The selected script is executed
- A loop ensures the menu keeps running until the user exits

This approach improves usability, as users do not need to run each script separately.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano main.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x main.sh
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 1: Menu-driven CLI Interface of the Project

1. Script 1 — System Information

Description

This script is designed to display important system-level information in a simple and readable format. It helps the user understand basic details about the system and the Python environment.

The script shows:

- Logged-in user
- Current working directory
- Current date and time
- System uptime
- Disk usage
- Installed Python version
- Python installation path

This script demonstrates the use of basic Linux commands and shell scripting concepts such as variables and command substitution.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/system_info.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/system_info.sh
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
1
```

```
========== System Information ==========
Logged-in User:
userbhagyashree
Current Working Directory:
/home/userbhagyashree/python-oss-audit
Date and Time:
Fri Mar 27 10:33:12 UTC 2026
System Uptime:
 10:33:12 up  1:21,  1 user,  load average: 0.00, 0.00, 0.00
----------------------------------------
Python Version:
Python 3.12.3
Python Executable Path:
/usr/bin/python3
----------------------------------------
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
none            3.8G     0  3.8G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.8G  4.0K  3.8G   1% /mnt/wsl
drivers         953G  186G  767G  20% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.9G  954G   1% /
========================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 2: Output Showing System and Python Information

2. Script 2 — File Operations

Description

This script performs basic file handling operations in Linux. It demonstrates how to create a file, write data into it, display its contents, copy it, and delete it.

These operations are useful in understanding how file management works in a Linux environment using shell scripting.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/file_ops.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scipts/file_ops.sh
chmod: cannot access 'scipts/file_ops.sh': No such file or directory
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/file_ops.sh
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
2
========== File Operations ==========
Creating a file named sample.txt...
Writing data into file...
Displaying file contents:
Hello, this is a sample file.
Copying file to sample_copy.txt...
Listing files:
README.md  main.sh  sample.txt  sample_copy.txt  scripts
Deleting original file...
Final file list:
README.md  main.sh  sample_copy.txt  scripts
=====================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 3: File Creation and File Operations

3. Script 3 — User Interaction

Description

This script demonstrates how a shell script can interact with the user by taking input and displaying personalized output.

The script asks the user to enter their name and their inspiration, and then prints a customized message. This shows how user input can be captured and used within a program.

.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/user_interaction.sh
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/user_interaction.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/user_interaction.py
chmod: cannot access 'scripts/user_interaction.py': No such file or directory
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/user_interaction.sh
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
3
========== User Interaction ==========
Enter your name:
Bhagyashree
Hello, Bhagyashree!
Who is your inspiration?
Abdul Kalam Sir
Abdul Kalam Sir was a great man.
======================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 4: User Input and Personalized Output

4. Script 4 — Process Check

Description

This script is used to display currently running processes in the system and filter out processes related to Python.

It helps in monitoring system activity and understanding how specific programs (like Python) are running in the background.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/process_check.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/process_check.sh
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
4
========== Process Check ==========
Top running processes:
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  21980 12544 ?        Ss   09:14   0:02 /sbin/init
root           2  0.0  0.0   3120  1920 ?        Sl   09:14   0:00 /init
root           6  0.0  0.0   3168  2036 ?        Sl   09:14   0:01 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          55  0.0  0.1  50436 15344 ?        S<s  09:15   0:00 /usr/lib/systemd/systemd-journald
-----------------------------------
Checking if Python is running:
root         239  0.0  0.2 107024 22272 ?        Ssl  09:15   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
userbha+    2540  0.0  0.0   4088  1920 pts/0    S+   11:04   0:00 grep python
==================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 5: Display of Running Processes

.

5. Script 5 — Automation Task

Description

This script demonstrates automation using shell scripting by creating a log file and storing important system information in it.

The script automatically records details such as:

- Current date and time
- Logged-in user
- Current working directory
- Python version

This reduces manual effort and shows how repetitive tasks can be automated efficiently.

```
userbhagyashree@Ayushi:~/python-oss-audit$ nano scripts/automation.sh
userbhagyashree@Ayushi:~/python-oss-audit$ chmod +x scripts/automation.sh
userbhagyashree@Ayushi:~/python-oss-audit$ /main.sh
bash: /main.sh: No such file or directory
userbhagyashree@Ayushi:~/python-oss-audit$ ./main.sh
==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
5
========== Automation Task ==========
Creating a log file...
Log file created successfully.
Displaying log file contents:
System Log - Fri Mar 27 11:08:34 UTC 2026
User: userbhagyashree
Directory: /home/userbhagyashree/python-oss-audit
Python Version: Python 3.12.3
====================================

==========================
 Python OSS Audit CLI Project
==========================
1. System Information
2. File Operations
3. User Interaction
4. Process Check
5. Automation Task
6. Exit
Enter your choice:
6
Exiting...
userbhagyashree@Ayushi:~/python-oss-audit$
```

Figure 6: Automated Log File Generation

Conclusion

The study of Python as an open-source software highlights its importance as a versatile and widely used programming language in modern computing. Its integration with Linux systems, along with its large ecosystem of libraries and frameworks, shows that Python is not only a development tool but also an essential part of system-level operations. The way Python is organized within the Linux file system and its interaction with users and permissions reflects the efficiency and flexibility of open-source environments.

Python also plays a key role in the modern software ecosystem. Its ability to work across different domains such as web development, data science, and automation makes it highly adaptable. The comparison with traditional architectures shows that Python provides more flexibility and supports modern development practices like modularity and scalability.

Although Python has some limitations, such as lower performance compared to low-level languages, its advantages like simple syntax, faster development, and strong community support make it more practical for most real-world applications. It is easy for beginners to learn while still being powerful enough for complex projects.

In conclusion, Python is a successful example of open-source software that combines simplicity, flexibility, and continuous improvement. Its growing popularity and widespread use indicate that it will continue to play an important role in the future of software development.